

Modelagem, abstração e modelos de banco de dados

Um banco de dados não é apenas um conjunto de arquivos. Ele precisa representar informações do mundo real de maneira estruturada.

Exemplo

Uma escola precisa controlar:

- Alunos
- Professores
- Disciplinas
- Turmas
- Matrículas
- Notas

Antes de criar tabelas ou escrever comandos SQL, é necessário entender:

- Quais informações precisam ser armazenadas
- Como elas se relacionam
- Quais regras devem ser respeitadas

Essa etapa é chamada de **modelagem de dados**.

O que é modelagem de dados?

A **modelagem de dados** é o processo de analisar uma realidade e representá-la de forma organizada em um banco de dados.

Ela permite transformar situações do mundo real em elementos que o sistema consegue armazenar e manipular.

Exemplo

Na realidade, temos:

Rodrigo está matriculado na disciplina de Banco de Dados.

Na modelagem, podemos representar:

- uma entidade **Aluno**;
- uma entidade **Disciplina**;
- uma relação **Matrícula** entre aluno e disciplina.

A modelagem ajuda a responder perguntas como:

- Quais objetos existem?
- Quais informações precisamos guardar sobre eles?
- Como esses objetos se relacionam?
- Quais regras precisam ser respeitadas?
- Que informação pode identificar cada registro?

Etapas gerais da modelagem

A modelagem costuma ser apresentada em três níveis principais:

Modelo conceitual

Representa a realidade de maneira mais próxima do negócio e menos dependente de tecnologia.

Nesse momento, ainda não é necessário definir tipos de dados ou comandos SQL.

Modelo lógico

Transforma o modelo conceitual em uma estrutura compatível com um determinado modelo de banco de dados, geralmente o relacional.

Exemplo:

- Entidade Aluno torna-se uma tabela
- Atributo nome torna-se uma coluna
- Relacionamento entre aluno e curso pode gerar uma chave estrangeira ou uma tabela intermediária

Modelo físico

Define como os dados serão efetivamente armazenados no SGBD.

Inclui decisões como:

- Tipos De Dados
- Índices
- Chaves
- Restrições
- Particionamento
- Detalhes De Armazenamento

O que é abstração de dados?

Abstração de dados é o processo de esconder detalhes complexos e apresentar apenas as informações relevantes para cada pessoa ou aplicação.

Um banco de dados possui muitos detalhes internos que não precisam ser conhecidos por todos os usuários.

Exemplo de abstração

Quando uma pessoa consulta o saldo bancário pelo aplicativo, ela visualiza:

- Saldo
- Extrato
- Transferências
- Pagamentos

Ela não precisa saber:

- Em qual servidor os dados estão
- Como os registros são organizados no disco
- Quais índices foram utilizados
- Como o banco encontrou o saldo

Essa separação torna o sistema mais simples de usar e manter.

Níveis de abstração

Nível externo ou de visão

É a forma como cada usuário ou aplicação visualiza os dados.

Exemplo:

- O aluno visualiza suas notas
- O professor visualiza as turmas
- A secretaria visualiza matrículas

Cada grupo pode ter uma visão diferente do mesmo banco.

Nível conceitual

Representa a estrutura geral do banco de dados.

Mostra:

- Quais entidades existem
- Quais atributos elas possuem
- Como se relacionam
- Quais regras gerais devem ser respeitadas

Nível interno ou físico

Representa como os dados são armazenados internamente.

Inclui:

- Arquivos
- Páginas de armazenamento
- Índices
- Estruturas físicas
- Organização dos registros

Independência de dados

A abstração possibilita a **independência de dados**.

Isso significa que alterações em um nível não devem obrigatoriamente exigir alterações nos demais níveis.

Independência física

É possível alterar a forma de armazenamento sem modificar a visão dos usuários.

Exemplo:

- Criar um índice
- Trocar a organização dos arquivos
- Melhorar o desempenho do armazenamento

Independência lógica

É possível alterar parte da estrutura lógica sem modificar todas as aplicações que acessam o banco.

Exemplo:

- Adicionar uma coluna
- Criar uma nova tabela
- Reorganizar determinadas informações

O que é o modelo relacional?

O **modelo relacional** organiza os dados em estruturas chamadas **relações**, normalmente representadas por tabelas.

É o modelo utilizado por bancos como:

- PostgreSQL
- MySQL
- MariaDB
- Oracle Database
- SQL Server
- SQLite

Tabela

Uma tabela representa uma **entidade** ou um conjunto de informações relacionadas.

Exemplo: tabela aluno.

id_aluno	nome	email
1	Ana Souza	ana@email.com
2	Carlos Lima	carlos@email.com
3	Marina Alves	marina@email.com

Linha ou tupla

Cada **linha**, também chamada de **tupla**, representa uma ocorrência ou registro.

Na tabela anterior:

- A primeira linha representa Ana
- A segunda linha representa Carlos
- A terceira linha representa Marina

Coluna ou atributo

Cada **coluna**, também chamada de **atributo**, representa uma característica dos registros.

Na tabela aluno:

- id_aluno identifica o aluno
- nome armazena o nome
- email armazena o endereço eletrônico

Domínio

O **domínio** é o conjunto de valores permitidos para um atributo.

Exemplos:

- idade: números inteiros positivos
- nota: valores entre 0 e 10
- estado civil: valores previamente definidos
- data de nascimento: datas válidas

Definir domínios ajuda a evitar dados inválidos.

Chave primária

A **chave primária** é o atributo, ou conjunto de atributos, utilizado para identificar unicamente cada linha de uma tabela.

Características

Uma chave primária deve ser:

- Única
- Obrigatória
- Não Nula.

Exemplo

id_aluno	nome
1	Ana Souza
2	Carlos Lima

Nesse exemplo, id_aluno é a chave primária.

Não seria adequado utilizar apenas o nome, pois duas pessoas podem ter o mesmo nome.

Chave primária composta

Em alguns casos, a identificação depende de mais de uma coluna.

Exemplo: tabela matricula.

id_aluno	id_disciplina	semestre
1	10	2026.1
1	11	2026.1

A combinação de id_aluno e id_disciplina pode formar uma chave primária composta.

Chave estrangeira

A **chave estrangeira** é uma coluna que estabelece uma relação com a chave primária de outra tabela.

Tabela aluno

id_aluno	nome
1	Ana Souza
2	Carlos Lima

Tabela matricula

id_matricula	id_aluno	data_matricula
101	1	10/02/2026
102	2	11/02/2026

Na tabela matricula, o campo id_aluno é uma chave estrangeira que referencia aluno.id_aluno.

Integridade referencial

A chave estrangeira ajuda a garantir que uma matrícula esteja associada a um aluno existente.

Não deveria ser possível inserir:

id_matricula	id_aluno
103	999

caso o aluno 999 não exista na tabela aluno.

Essa regra é chamada de **integridade referencial**.

Restrições de integridade

As **restrições** são regras aplicadas pelo banco de dados para impedir informações inválidas ou inconsistentes.

PRIMARY KEY

Define a chave primária.

Exemplo conceitual:

id_aluno: identifica unicamente o aluno

FOREIGN KEY

Define uma chave estrangeira e uma relação entre tabelas.

NOT NULL

Indica que uma coluna deve obrigatoriamente receber um valor.

Exemplo:

- O nome de um aluno não pode ficar vazio.

UNIQUE

Impede valores repetidos em uma coluna ou combinação de colunas.

Exemplo:

- Dois usuários não podem ter o mesmo e-mail.

CHECK

Define uma condição que deve ser obedecida.

Exemplo:

$$0 \leq \text{nota} \leq 10$$

DEFAULT

Define um valor padrão quando nenhum valor é informado.

Exemplo:

- Uma data de cadastro pode receber automaticamente a data atual
- Um pedido pode iniciar com o status "Pendente"

Integridade de entidade

Determina que cada linha deve ser identificada por uma chave primária válida.

Integridade referencial

Determina que uma chave estrangeira deve apontar para um registro existente, salvo regras específicas de exclusão ou atualização.

Integridade de domínio

Determina que os valores armazenados devem respeitar o tipo e as regras do atributo.

O que é normalização?

A **normalização** é um conjunto de técnicas utilizadas para organizar os dados e reduzir problemas de redundância e inconsistência.

Ela ajuda a:

- Evitar repetição desnecessária
- Reduzir erros de atualização
- Facilitar a manutenção
- Melhorar a organização das tabelas
- Preservar a integridade dos dados

Problema: tabela mal organizada

Imagine uma tabela de pedidos:

id_pedido	cliente	cidade	produto_1	produto_2
1	Ana	Recife	Teclado	Mouse

Essa estrutura apresenta problemas:

- Quantidade fixa de produtos
- Dificuldade para adicionar novos produtos
- Informações repetidas
- Dificuldade para consultar cada item
- Possibilidade de campos vazios

Uma organização melhor separaria as informações:

Tabela cliente

id_cliente	nome	cidade
1	Ana	Recife

Tabela pedido

id_pedido	id_cliente
1	1

Tabela produto

id_produto	descricao
10	Teclado
11	Mouse

Tabela item_pedido

id_pedido	id_produto	quantidade
1	10	1
1	11	1

Anomalias que a normalização procura evitar

Anomalia de inserção

Não é possível cadastrar uma informação sem cadastrar outra que não deveria ser obrigatória.

Exemplo:

- Não conseguir cadastrar um produto porque ainda não existe nenhum pedido para ele.

Anomalia de atualização

A mesma informação aparece em vários lugares e precisa ser atualizada repetidamente.

Anomalia de exclusão

Ao excluir um registro, informações importantes são perdidas acidentalmente.

Formas normais

As **formas normais** são etapas utilizadas para verificar se uma estrutura de banco de dados está bem organizada. Elas ajudam a reduzir:

- Repetição de informações
- Inconsistências
- Dificuldades de atualização
- Problemas na inserção e exclusão de dados

A normalização deve ser analisada considerando as **dependências entre os atributos** e o significado de cada informação.

Normalizar não significa simplesmente criar muitas tabelas. Significa colocar cada informação no local mais adequado, evitando que uma coluna dependa de algo que não deveria ser sua referência.

Primeira Forma Normal - 1FN

Uma tabela está na **Primeira Forma Normal** quando:

- Cada coluna contém valores indivisíveis
- Não existem listas armazenadas em uma única célula
- Não existem grupos repetidos de colunas
- Cada linha representa um registro

Exemplo fora da 1FN

Considere a tabela aluno:

id_aluno	nome	telefones
1	Ana Souza	99999-1111, 98888-2222
2	Carlos Lima	97777-3333

A coluna telefones possui mais de um valor na mesma célula. Isso dificulta:

- Pesquisar um telefone específico
- Alterar apenas um dos telefones
- Excluir um telefone
- Validar os valores
- Relacionar telefones com outras informações

Transformação para a 1FN

Podemos criar uma tabela específica para os telefones:

Tabela aluno

id_aluno	nome
1	Ana Souza
2	Carlos Lima

Tabela telefone_aluno

id_telefone	id_aluno	telefone
1	1	99999-1111
2	1	98888-2222
3	2	97777-3333

Agora cada célula contém apenas um valor, e cada telefone pode ser manipulado individualmente.

Outro exemplo: grupos repetidos

Uma estrutura inadequada seria:

id_pedido	produto_1	produto_2	produto_3
1	Teclado	Mouse	Monitor

Esse modelo apresenta uma quantidade fixa de colunas para produtos. Se o pedido tiver quatro produtos, será necessário criar uma nova coluna. Se tiver apenas um, haverá campos vazios.

Uma estrutura compatível com a 1FN seria:

Tabela pedido

id_pedido	data_pedido
1	16/08/2026

Tabela item_pedido

id_pedido	numero_item	produto	quantidade
1	1	Teclado	1
1	2	Mouse	2
1	3	Monitor	1

Cada linha representa um item do pedido.

Observação:

Existem diversas formas normais utilizadas na normalização de bancos de dados.

Entre as mais conhecidas estão a Primeira Forma Normal (1FN), a Segunda Forma Normal (2FN) e a Terceira Forma Normal (3FN).

Também existem formas normais mais avançadas, como a Forma Normal de Boyce-Codd (BCNF), a Quarta Forma Normal (4FN) e a Quinta Forma Normal (5FN).

Inicialmente, porém, nosso foco será apenas na Primeira Forma Normal (1FN). Ela é o ponto de partida para a organização das tabelas e estabelece regras básicas.